Strings

Overview

A string is a sequence of characters.

Strings are:

- ordered
- immutable
- index-based

That means you can read characters by position, but you cannot directly change the original string after it is created.

Topic: String Slicing

Definition

String slicing lets you read multiple characters at once.

Syntax

```
my_string[start:end]
```

This creates a new string containing characters from:

- `start`
- up to `end - 1`

Example

```
my_string = "Armando"
name = my_string[0:3]
print(name)
```

Output:

```
Arm
```

Important Notes

- Lists and tuples also support slicing
- The ending index is not included
- Python creates a new string object when slicing
- Changing the original string later does not change the slice

---

Omitting Start or End

Omit the start index

```
my_str[:3]
```

This returns characters from index `0` up to `2`.

Omit the end index

```
my_str[4:]
```

This returns characters from index `4` to the end of the string.

Negative Indexes

Negative indexes count from the end of the string.

Example

```
my_string = "Armando"
name = my_string[0:-2]
```

```
print(name)
```

Output:

```
Arman
```

Slice Stride

A slice can include a third value called the stride.

Syntax

```
my_string[start:end:stride]
```

Notes

- Stride tells Python how much to move forward after each character
- Default stride is `1`

Example

```
numbers = "0123456789"

print(numbers[::])
print(numbers[::2])
print(numbers[1:9:3])
```

Output:

```
0123456789
02468
147
```

Combining Slices

You can connect slices using `+`.

Example

```
my_name = "Armando"
name = my_name[:1] + my_name[-1:]
print(name)
```

Output:

```
Ao
```

Common Slicing Operations

Given:

```
my_str = "http://en.wikipedia.org/wiki/Nasa/"
```

Expression	Result	Description
`my_str[10:19]`	`wikipedia`	Returns characters from index 10 to 18
`my_str[10:-5]`	`wikipedia.org/wiki/`	Returns characters from index 10 to 28
`my_str[8:]`	`n.wikipedia.org/wiki/Nasa/`	Returns all characters from index 8 to the end
`my_str[:23]`	`http://en.wikipedia.org`	Returns all characters up to, but not including, index 23
`my_str[:-1]`	`http://en.wikipedia.org/wiki/Nasa`	Returns all characters except the last one

Topic: Advanced String Formatting

Definition

Sometimes `print(x)` is not enough.

String formatting helps control:

- spacing
- alignment
- fill characters
- decimal precision

Field Width

Field width sets the minimum number of characters used in the output.

Example

```
print(f"{'Player Name':16}{'Goals':8}")
```

Notes

- Strings are left-aligned by default
- Numbers are right-aligned by default
- Extra spaces are added if needed

Alignment Characters

You can control alignment inside the field width.

Alignment	Symbol
Left	`<`
Right	`>`
Center	`^`

Examples

```
f"{'Player Name':<16}{'Goals':<8}"
f"{'Player Name':>16}{'Goals':>8}"
f"{'Player Name':^16}{'Goals':^8}"
```

Fill Characters

A fill character pads the extra space in a field.

Notes

- Default fill is a blank space
- A custom fill character goes before the alignment symbol

Examples

Format	Value	Output
`{score:}`	`9`	`9`
`{score:4}`	`9`	`9`
`{score:0>4}`	`9`	`0009`
`{score:0>4}`	`18`	`0018`
`{score:0^4}`	`18`	`0180`

Floating-Point Precision

Precision controls how many digits appear after the decimal point.

Example

```
print(f"{1.725:.1f}")
```

Output:

```
1.7
```

Notes

- If precision is larger than the available digits, Python adds trailing zeros
- If precision is smaller, Python rounds the number

Topic: String Methods

Overview

Strings have many built-in methods for useful operations like:

- replacing text
- changing letter case
- searching for substrings
- cleaning up whitespace

Important

Strings are immutable, so methods return a new string instead of changing the original directly.

Finding and Replacing

`replace(old, new)`

Returns a copy of the string with all matches of `old` replaced by `new`.

`replace(old, new, count)`

Same as above, but only replaces the first `count` matches.

Example

```
phrase = "My name is Armando"
phrase = phrase.replace("Armando", "Lena")
print(phrase)
```

Output:

```
My name is Lena
```

Finding the Position of a Character or Substring

`find(x)`

Returns the index of the first occurrence of `x`.

Returns `-1` if not found.

Examples

```
my_name = "Armando"
print(my_name.find("A"))
print(my_name.find("Arm"))
```

Output:

```
0
0
```

`find(x, start)`

Starts searching at index `start`.

```
my_name = "Armando"
print(my_name.find("a", 2))
```

Output:

```
3
```

`find(x, start, end)`

Searches from `start` up to `end - 1`.

```
my_name = "ArmandoLena"
print(my_name.find("a", 4, len(my_name) - 1))
```

`rfind(x)`

Searches from the end and returns the last occurrence.

`count(x)`

Returns how many times `x` appears in the string.

Note About `find()` vs `in`

Use:

- `find()` or `rfind()` when you need the exact position
- `in` when you only need to know whether something exists

Better containment check

```
if "batman" in superhero_name:
    print("Found batman")
```

Important correction

Do not use `is` for substring checking.

Wrong:

```
if "batman" is superhero_name:
```

Right:

```
if "batman" in superhero_name:
```

Topic: Comparing Strings

Strings are compared character by character.

Python compares them using character values behind the scenes.

Examples

Example	Result	Why
`"Hello" == "Hello"`	`True`	Both strings are exactly the same
`"Hello" == "Hello!"`	`False`	The second string has an extra `!`
`"Yankee Sierra" > "Amy Wise"`	`True`	`"Y"` is greater than `"A"`
`"Yankee Sierra" > "Yankee Zulu"`	`False`	The strings match until `"S"` is compared with `"Z"`
`"seph" in "Joseph"`	`True`	`"seph"` appears inside `"Joseph"`
`"jo" in "Joseph"`	`False`	Lowercase `"j"` is different from uppercase `"J"`

Note

If one string is shorter and both match up to that point, the shorter string is considered smaller.

Membership Operators

Membership operators check whether a substring exists in a string.

- `in`
- `not in`

Example

```
"cat" in "category"
```

Identity Operators

Identity operators check whether two variables refer to the same object.

- `is`
- `is not`

Common Error

Do not use `is` when comparing string values.

Wrong:

```
name is "Armando Sarza"
```

Right:

```
name == "Armando Sarza"
```

Topic: String Boolean Methods

These methods return `True` or `False`.

Method	Meaning
`isalnum()`	`True` if all characters are letters or numbers
`isdigit()`	`True` if all characters are digits
`islower()`	`True` if all cased characters are lowercase
`isupper()`	`True` if all cased characters are uppercase
`isspace()`	`True` if all characters are whitespace
`startswith(x)`	`True` if the string starts with `x`
`endswith(x)`	`True` if the string ends with `x`

Topic: Creating New Strings from a String

These methods return modified copies of a string.

Method	Description
`capitalize()`	First character uppercase, rest lowercase
`lower()`	All characters lowercase
`upper()`	All characters uppercase
`strip()`	Removes leading and trailing whitespace
`title()`	Capitalizes the first letter of each word

Topic: Splitting and Joining Strings

Purpose

These methods help break apart strings or combine them again.

`split()` Method

Definition

`split()` breaks a string into a list of smaller strings called tokens.

Example

```
"My name is Armando".split()
```

Output:

```
["My", "name", "is", "Armando"]
```

Example with a separator

```
"My,name,is,Lena".split(",")
```

Output:

```
["My", "name", "is", "Lena"]
```

Input example

```
my_list = input().split()
```

Notes

- The separator is what tells Python where to split
- If no separator is given, Python splits on whitespace
- If the string starts or ends with a separator, or has repeated separators, empty strings can appear in the list
- When splitting by whitespace, Python does not create empty strings from extra spaces

---

Inserting into a Split List

After splitting, you can insert into the list.

Example

```
list_created.insert(index, "word")
```

`join()` Method

Definition

`join()` does the opposite of `split()`.

It joins a list of strings into one single string.

Example

```
user_name = ["My", "name", "is", "Armando"]
separator = "+"
name = separator.join(user_name)
print(name)
```

Output:

```
My+name+is+Armando
```

Shorter version

```
intro_message = "+".join(user_name)
```

Notes

- `join()` is useful for building strings with separators
- It is also useful for combining strings without separators
- `join()` is usually cleaner and easier to read than building a string with loops

---

Quick Summary

Main ideas

- Strings are immutable
- Slicing reads parts of a string
- Formatting controls spacing, alignment, fill, and precision
- String methods help search, replace, clean, and transform text
- `split()` breaks a string into a list
- `join()` combines a list into a string

Important reminders

- Use `==` to compare values
- Use `is` only for object identity
- Use `in` to check whether a substring exists
- String methods return new strings, they do not change the original